→ If you have not noticed yet, our dataset is indeed imbalance. We have way more emails that are not spam than emails that are spam.

→ Accuracy can sometimes hide the nuance of imbalanced datasets. The reason is in these kinds of datasets, the true negative category dominates, diluting the effect of the rest.

So, even if the classifier were to perform poorly in the other 3 categories, its Accuracy will still look good, masking its deficiencies.

TN (14)	FP (3)
FN (2)	TP (1)

The True Negatives dominate the Accuracy calculation.

⇒ Precision, Recall & F1 can provide a more balanced view of a classifier's performance because they don't focus on True Negative (TN) in calculation.

⇒ B/w Precision & Recall though, there is a tradeoff. It is hard to optimize for both simultaneously as optimizing for the FP (thereby improving Precision) comes at the expense of the ~~FP~~ FN (thereby deteriorating Recall) & vice versa.
(False Negative)

⇒ Which then brings the question: which metric should you prioritize – Precision or Recall?

The answer is that it depends on the nature of your task. Let's see why.

Suppose the spam classifier achieved high Precision & low Recall (Scenario A). This would result in fewer non-spam emails flagged as spam (FP). But this would also mean more of the actual spam emails went undetected (FN).

Conversely, if the classifier achieved high Recall & low Precision (Scenario B), there would be fewer undetected spam emails (FN), but it comes at the expense of more non-spam emails being flagged as spam (FP).

For a spam classification task, it's probably more desirable to avoid important emails being moved into the spam folder than to have the occasional spam emails going into the Inbox. So, for this task, we will want to prioritize Precision over Recall.

↳ If both Precision & Recall are important to you & you need the classifier to do well in both, Then use F1 score. F1 provides the balance b/w Precision & Recall.

⇒ Multi-Class Classification

Let's say we have the task of classifying a list of eCommerce customer emails into one of the 3 classes: Shipping, Returns & Tracking. We'll represent each class with integer values of 0, 1 & 2 respectively.

The dataset contains 15 email titles. We put each data point through a multi-class classifier to

get the predicted class & then compare it with its actual class.

First, Accuracy. The calculation is the same as we did with binary classification – the no. of correct predictions divided by total no. of predictions. For our dataset, there are 10 correct predictions & 5 incorrect predictions, which give us an Accuracy of 67%.

Now, to compute Precision, Recall & F1, we'll build the Confusion matrix.

Since, we have 3 classes, the matrix now becomes a 3×3 matrix, each column representing one class. Applied to our dataset, we get the following matrix.

→ Prediction

Actual ↓	0 Shipping	1 Returns	2 Tracking
0 Shipping	TRUE 3	FALSE 0	FALSE 2
1 Returns	FALSE 1	TRUE 4	FALSE 0
2 Tracking	FALSE 0	FALSE 2	TRUE 3

Precision & Recall are calculated in the same way as we did with binary classification. The only difference now is each class will have its own set of Precision & Recall values.

Let's take class 0 (shipping) as an example. Here, we use Positive to refer to the shipping class & Negative to refer to all the other classes (Not shipping).

So, Precision (For Class 0 = shipping) $= \frac{TP}{TP+FP} = \frac{3}{3+1+0} = 75\%$

Recall (" " " ") $= \frac{TP}{TP+FN} = 60\%$

F1 (" " " ") $= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 67\%$

↳ Going through all classes in our dataset, this is what Recall, Precision & F1 look like

Class	Precision	Recall	F1
0-Shipping	75%	60%	67%
1-Returns	67%	80%	79%
2-Tracking	60%	60%	53%

↳ Now, what if the no. of classes started to get really big? This table would become huge & it will be harder to get a snapshot of the performance. So, we may want to still have a single value for each of these metrics.

↳ There are a few approaches we can take & one common option is to take the average of all the classes. This is also called the Macro-Average, and we apply it to each of Precision, Recall & F1 as follows:

$$\text{Macro-precision} = \frac{75\% + 67\% + 60\%}{3} = 67\%$$

$$\text{Macro-Recall} = \frac{60\% + 80\% + 60\%}{3} = 67\%$$

$$\text{Macro-F1} = \frac{67\% + 79\% + 53\%}{3} = 66\%$$

☒ Few-Shot

Here you'll le
by their sent
classify endp

↳ In the text
developers & te
AI-based classi
from scratch or
to have the
requires a hu
build a wo

↳ With LLMs,
training data
a handful of
classification. T
costs, time &
This means t
deploying t
they would no

↳ Let's see ho
the sentiment
or neutral. Th
media content

Few-Shot Classification

Here you'll learn how to classify a small dataset of sentences by their sentiment (positive, negative or neutral), using Cohere's classify endpoint.

↳ In the text classification space, a trend is emerging where developers & teams are leveraging LLMs when building an AI-based classifier system. This is opposed to building a system from scratch on their own, which first, requires the team to have the know-how in ML & engineering & second, requires a huge amount of labeled training data to build a working solution.

↳ With LLMs, instead of having to prepare thousands of training data points, you can get up & running with just a handful of ~~exp~~ examples per class, called few-shot classification. Think about the sudden drop in the associated costs, time & effort to collect & build a training dataset. This means that more teams can now think about deploying their own text classification systems, whereas they would not have considered it before.

↳ Let's see how to do this with an example. We'll classify the sentiment of text into a no. of classes, say, positive, negative or neutral. This is useful for app?s like analyzing social media content or categorizing product feedback.

For example, a human can easily tell you that, "Hello, World! What a beautiful day" conveys a positive sentiment, but let's see if our models can do that too. And while we're at it, let's try classifying other phrases that you might find on social media.

Here, you'll learn to classify text based on sentiment using Cohere's classify endpoint.

↳ Step-by-Step Guide

```
import cohere
from cohere import ClassifyExample
#create a cohere client
co = cohere.Client("Cohere-Api-key")
```

↳ Step ① Prepare Examples & Input

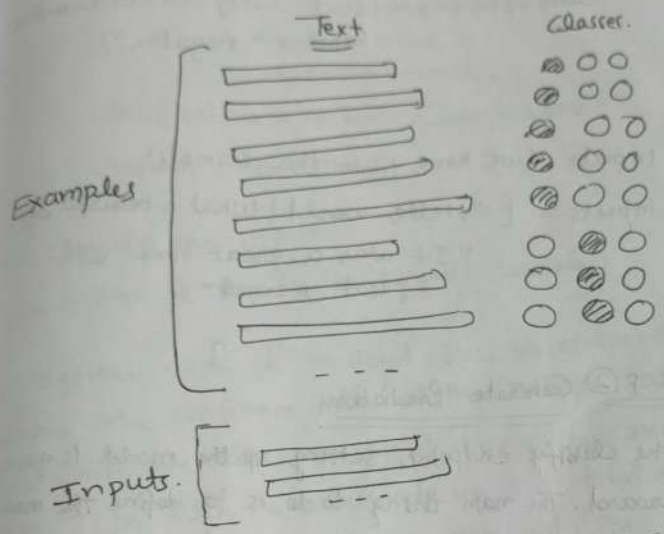
A typical ML model requires many training examples to perform text classification, but with the Classify endpoint, you can get started with as few as two examples per class. You need to prepare the following:

Examples

- These are the training examples we give the model to show the output we want it to generate.
- Each example contains the text itself & the corresponding label, or class.
- The minimum no. of examples required is two per class
- You can have as many classes as possible. If you are classifying text into 2 classes that means you need a min. of 4 examples & so on.

Inputs.

- These are list of text pieces you'd like to classify



Our sentiment analysis classifier has 3 classes with 5 examples each: "Positive" for a +ve sentiment, "Negative" for a -ve sentiment & "Neutral" for a neutral sentiment. The code looks as follows:

The examples:

```
examples = [ ClassifyExample(text="I'm so proud of you",
                             label="positive"),
             ClassifyExample(text="What a great time to
                             be alive", label="positive")
             ClassifyExample(text="That's awesome work",
                             label="positive"),
```

```
ClassifyExample( text= "The service was amazing",
                 label = "positive"),
ClassifyExample(text= "They don't care about me",
                 label = "negative"),
```

- - - -

The inputs (we have 12 in this example):

```
inputs = [ "Hello, world! What a beautiful day",
           "It was a great time with
                great people",
           - - -
           . . . ]
```

↳ Step ② Generate Predictions

With the classify endpoint, setting up the model is quite straightforward. The main thing to do is to define the model type. For the classify endpoint, we need to use an embedding model, & we'll use "embed-english-v3.0"

Putting everything together with the classify endpoint look like

```
→ def classify_text(inputs, examples):
    '''
    classifies a list of input texts given the examples.
    Arguments:
        model(str): identifier of the model
        inputs(list[str]): a list of input texts to be classified
        examples(list[Example]): a list of example texts & class
                                 labels
    Returns:
        classifications(list): each result contains the text, labels &
                               conf values
    '''
```

```
# Classify text by calling the classify endpoint
response = co.classify(
                model = 'embed-english-v3.0',
                inputs = inputs,
                examples = examples)
classifications = response.classifications
return classifications.

# classify the inputs
predictions = classify-text(inputs, examples)
```

↳ Together with the predicted class, the endpoint also returns the confidence value of the prediction (b/w 0 & 1). These confidence values are split among the classes, in this case 3, in which the values add up to a total of 1. The classifier then selects the class with the highest confidence value as the "predicted class". A high confidence value for the predicted class therefore indicates that the model is very confident of its prediction & vice-versa.

↳ Here's a sample output returned:

Input: Hello, world | What a beautiful day
Prediction: positive
Confidence: 0.84

Input: It was a great time with great people
Prediction: positive
Confidence: 0.99